

REPORTE DE PRÁCTICA: SEMÁFORO CON FSM

Control Vehicular y Peatonal con Máquina de Estados Finitos y Temporización No Bloqueante

Asignatura: Sistemas Programables **Plataforma:** Arduino UNO R3 / R4 **Práctica:** Laboratorio 3
Entorno: Tinkercad / Arduino IDE **Paradigma:** FSM (enum class) **Estado:** Validado y Operativo

1. Descripción del Proyecto

El presente proyecto aborda el diseño e implementación del sistema de control para un **semáforo vehicular y peatonal interactivo**. El núcleo del desarrollo se fundamenta en una **Máquina de Estados Finitos (FSM)** programada en C++ para Arduino. La solución permite la alternancia secuencial entre los estados típicos de tráfico vehicular (Verde, Amarillo y Rojo) e integra una solicitud peatonal asíncrona gestionada mediante un pulsador con filtro antirrebote (*debounce*) implementado por software.

Toda la gestión temporal del sistema se ejecuta de forma **no bloqueante** mediante la función nativa `millis()`, eliminando por completo el uso de retardos bloqueantes como `delay()`. Esto garantiza que el microcontrolador mantenga una capacidad de respuesta continua para la lectura de entradas y el monitoreo serial en tiempo real.

2. Objetivos de Aprendizaje

- **Modelado de Sistemas Secuenciales:** Diseñar una arquitectura FSM clara, estructurada e inmutable mediante construcciones fuertemente tipadas usando `enum class`.
- **Programación Concurrente y No Bloqueante:** Reemplazar la temporización imperativa por contadores basados en `millis()` para permitir multitarea reactiva.
- **Filtrado de Entradas Digitales:** Aplicar algoritmos de antirrebote (*debounce*) por software para mitigar el ruido mecánico producido por pulsadores.
- **Lógica de Prioridad y Seguridad Vial:** Coordinar los tiempos de transición garantizando que una solicitud peatonal no interrumpa abruptamente el flujo vehicular, asegurando una fase amarilla previa.

3. Materiales Utilizados

COMPONENTE	CANTIDAD	ESPECIFICACIÓN / FUNCIÓN TÉCNICA
Placa Arduino UNO	1 unidad	Microcontrolador ATmega328P / Renesas RA4M1 para la ejecución del código.
LED Vehicular Rojo	1 unidad	Señalización de detención vehicular (Conectado a Pin Digital 8).
LED Vehicular Amarillo	1 unidad	Señalización de prevención vehicular (Conectado a Pin Digital 9).
LED Vehicular Verde	1 unidad	Señalización de paso vehicular (Conectado a Pin Digital 10).
LED Peatonal Rojo	1 unidad	Prohibición de cruce peatonal (Conectado a Pin Digital 6).
LED Peatonal Verde	1 unidad	Autorización de cruce peatonal (Conectado a Pin Digital 7).

COMPONENTE	CANTIDAD	ESPECIFICACIÓN / FUNCIÓN TÉCNICA
Resistencias 220 Ω	5 unidades	Limitadoras de corriente conectadas entre el cátodo de cada LED y GND.
Pulsador (Pushbutton)	1 unidad	Solicitud de paso peatonal (Conectado a Pin Digital 2 con INPUT_PULLUP).
Protoboard y Jumpers	1 kit	Infraestructura de interconexión y líneas comunes de alimentación/ tierra.

4. Diagrama y Mapeo de Conexiones

Las conexiones del circuito respetan la topología de limitación de corriente por el lado del cátodo hacia el bus común de tierra (GND). El pulsador utiliza la resistencia interna de *pull-up* del microcontrolador, simplificando el cableado físico.

ELEMENTO	PIN ARDUINO	CONEXIÓN EN PROTOBOARD	CONFIGURACIÓN I/O
Pulsador Peatonal	PIN 2	Terminal a Pin 2; Terminal opuesto a bus GND	INPUT_PULLUP
LED Peatonal Rojo	PIN 6	Ánodo a Pin 6; Cátodo con R 220 Ω a GND	OUTPUT
LED Peatonal Verde	PIN 7	Ánodo a Pin 7; Cátodo con R 220 Ω a GND	OUTPUT
LED Vehicular Rojo	PIN 8	Ánodo a Pin 8; Cátodo con R 220 Ω a GND	OUTPUT
LED Vehicular Amarillo	PIN 9	Ánodo a Pin 9; Cátodo con R 220 Ω a GND	OUTPUT
LED Vehicular Verde	PIN 10	Ánodo a Pin 10; Cátodo con R 220 Ω a GND	OUTPUT

Nota de Diseño Eléctrico: La configuración de entrada INPUT_PULLUP en el Pin 2 mantiene la línea en un estado lógico ALTO (HIGH) de manera predeterminada. Al presionar el pulsador, la línea se conecta directamente a GND, generando un flanco de bajada (LOW) que es detectado de forma limpia por el código.

5. Análisis de la Máquina de Estados (FSM)

El comportamiento del sistema se estructuró a través del tipo enumerado fuertemente tipado `enum class EstadoSemaforo`. Los cuatro estados operativos y sus condiciones de transición se detallan a continuación:

ESTADO (FSM)	VEHICULAR	PEATONAL	DURACIÓN	CONDICIÓN DE SALIDA
VERDE_VEHICULO	VERDE	ROJO	5000 ms (mín.)	Transcurrido el tiempo mínimo Y registrada solicitud peatonal.

ESTADO (FSM)	VEHICULAR	PEATONAL	DURACIÓN	CONDICIÓN DE SALIDA
AMARILLO_VEHICULO	AMARILLO	ROJO	2000 ms	Cumplidos los 2000 ms de advertencia.
VERDE_PEATON	ROJO	VERDE	4000 ms	Cumplidos los 4000 ms de paso peatonal seguro.
PARPADEO_PEATON	ROJO	VERDE (INTERMITENTE)	2000 ms	Cumplidos los 2000 ms (destellos a 250 ms) \$ ightarrow\$ Regresa a VERDE_VEHICULO.

Lógica Anti-rebote (Debounce)

Para evitar lecturas falsas causadas por el rebote mecánico del pulsador, se implementó una ventana de validación temporal de **50 ms** (TIEMPO_DEBOUNCE). Solo si el pin permanece en estado LOW continuamente durante ese intervalo, se confirma la pulsación y se activa la bandera de solicitudPeaton = true.

6. Código Fuente Implementado

A continuación se presenta el código C++ completo ejecutado en la placa de desarrollo:

```

/*
  Práctica 3: Semáforo Vehicular y Peatonal con FSM (enum class)
  Desarrollado para Arduino UNO / Tinkercad
  Mapeo de Pines:
  - Vehicular: Rojo (8), Amarillo (9), Verde (10)
  - Peatonal:  Rojo (6), Verde (7)
  - Pulsador:  Pin 2 (INPUT_PULLUP)
*/

const uint8_t LED_VEH_ROJO      = 8;
const uint8_t LED_VEH_AMARILLO = 9;
const uint8_t LED_VEH_VERDE     = 10;

const uint8_t LED_PEAT_ROJO     = 6;
const uint8_t LED_PEAT_VERDE    = 7;

const uint8_t BOTON_PEATON      = 2;

enum class EstadoSemaforo {
  VERDE_VEHICULO,
  AMARILLO_VEHICULO,
  VERDE_PEATON,
  PARPADEO_PEATON
};

EstadoSemaforo estadoActual = EstadoSemaforo::VERDE_VEHICULO;

const unsigned long TIEMPO_VERDE_VEH = 5000;
const unsigned long TIEMPO_AMARILLO  = 2000;
const unsigned long TIEMPO_VERDE_PEAT = 4000;

```

```

const unsigned long TIEMPO_PARPADEO    = 2000;
const unsigned long INTERVALO_PARPADEO = 250;

unsigned long tiempoInicioEstado = 0;
unsigned long ultimoParpadeo = 0;
bool solicitudPeaton = false;

unsigned long ultimoDebounce = 0;
const unsigned long TIEMPO_DEBOUNCE = 50;
bool ultimoEstadoBoton = HIGH;

void setup() {
    pinMode(LED_VEH_ROJO, OUTPUT);
    pinMode(LED_VEH_AMARILLO, OUTPUT);
    pinMode(LED_VEH_VERDE, OUTPUT);

    pinMode(LED_PEAT_ROJO, OUTPUT);
    pinMode(LED_PEAT_VERDE, OUTPUT);

    pinMode(BOTON_PEATON, INPUT_PULLUP);

    Serial.begin(115200);
    Serial.println("--- Semáforo FSM Iniciado ---");

    tiempoInicioEstado = millis();
}

void loop() {
    unsigned long ahora = millis();

    // 1. Lectura del botón con Anti-rebote (Debounce)
    bool lecturaBoton = digitalRead(BOTON_PEATON);
    if (lecturaBoton != ultimoEstadoBoton) {
        ultimoDebounce = ahora;
    }
    if ((ahora - ultimoDebounce) > TIEMPO_DEBOUNCE) {
        if (lecturaBoton == LOW) {
            if (!solicitudPeaton) {
                solicitudPeaton = true;
                Serial.println("-> Solicitud peatonal registrada.");
            }
        }
    }
    ultimoEstadoBoton = lecturaBoton;

    // 2. Máquina de Estados Finita (FSM)
    switch (estadoActual) {

        case EstadoSemaforo::VERDE_VEHICULO:
            digitalWrite(LED_VEH_ROJO, LOW);
            digitalWrite(LED_VEH_AMARILLO, LOW);
            digitalWrite(LED_VEH_VERDE, HIGH);

            digitalWrite(LED_PEAT_ROJO, HIGH);
            digitalWrite(LED_PEAT_VERDE, LOW);

            if ((ahora - tiempoInicioEstado >= TIEMPO_VERDE_VEH) && solicitudPeaton) {
                estadoActual = EstadoSemaforo::AMARILLO_VEHICULO;
                tiempoInicioEstado = ahora;
                Serial.println("Estado: Amarillo Vehicular");
            }
            break;
    }

```

```

case EstadoSemaforo::AMARILLO_VEHICULO:
    digitalWrite(LED_VEH_VERDE, LOW);
    digitalWrite(LED_VEH_AMARILLO, HIGH);

    if (ahora - tiempoInicioEstado >= TIEMPO_AMARILLO) {
        estadoActual = EstadoSemaforo::VERDE_PEATON;
        tiempoInicioEstado = ahora;
        solicitudPeaton = false;
        Serial.println("Estado: Verde Peatonal");
    }
    break;

case EstadoSemaforo::VERDE_PEATON:
    digitalWrite(LED_VEH_AMARILLO, LOW);
    digitalWrite(LED_VEH_ROJO, HIGH);

    digitalWrite(LED_PEAT_ROJO, LOW);
    digitalWrite(LED_PEAT_VERDE, HIGH);

    if (ahora - tiempoInicioEstado >= TIEMPO_VERDE_PEAT) {
        estadoActual = EstadoSemaforo::PARPADEO_PEATON;
        tiempoInicioEstado = ahora;
        ultimoParpadeo = ahora;
        Serial.println("Estado: Parpadeo Peatonal");
    }
    break;

case EstadoSemaforo::PARPADEO_PEATON:
    if (ahora - ultimoParpadeo >= INTERVALO_PARPADO) {
        ultimoParpadeo = ahora;
        digitalWrite(LED_PEAT_VERDE, !digitalRead(LED_PEAT_VERDE));
    }

    if (ahora - tiempoInicioEstado >= TIEMPO_PARPADO) {
        estadoActual = EstadoSemaforo::VERDE_VEHICULO;
        tiempoInicioEstado = ahora;
        Serial.println("Estado: Verde Vehicular");
    }
    break;
}
}
}

```

7. Conclusiones

La realización de esta práctica permitió comprender la importancia crítica de desacoplar la temporización del flujo de ejecución del programa. El uso de la función `millis()` en sustitución de `delay()` eliminó los bloqueos del procesador, haciendo posible atender eventos externos (como la pulsación del botón) en cualquier microsegundo.

Asimismo, la estructura mediante `enum class` garantizó un código legible, modular y libre de errores de tipado implícito, facilitando la adición de nuevas fases como el parpadeo de advertencia peatonal. Finalmente, la integración del filtro de antirrebote asegura la estabilidad del sistema frente a ruido inductivo o mecánico en el pulsador, logrando un control vial confiable y eficiente.